



"Security-First Architecture"

Document Type:	Security & Access Control Document
Version:	v1.0
Date:	June 2026
Author:	Platform Developer
Classification:	Internal — Confidential

■ Table of Contents

- 1. Security Overview & Principles
- 2. Authentication & Authorization
- 3. Data Protection & Encryption
- 4. Input Validation & Sanitization
- 5. Rate Limiting & Abuse Prevention
- 6. API Security
- 7. Third-Party Integration Security
- 8. Compliance & Privacy
- 9. Incident Response Plan
- 10. Security Checklist

1. Security Overview & Principles

IdeaDNA handles sensitive user data including code snippets, project ideas, and personal information. The security architecture follows defense-in-depth principles with multiple layers of protection. All security measures are designed to be transparent to users while maintaining robust protection.

Security Principles

- Defense in Depth: Multiple security layers — no single point of failure
- Least Privilege: Users and services have minimum required access
- Zero Trust: Verify every request, never trust by default
- Data Minimization: Collect only necessary data, delete when no longer needed
- Transparency: Clear privacy policy, user control over data
- Proactive Monitoring: Continuous security scanning and alerting

2. Authentication & Authorization

2.1 Authentication Flow

- User clicks 'Sign Up' → Redirected to Clerk hosted UI
- User authenticates via OAuth (Google/GitHub) or email/password
- Clerk validates credentials and issues JWT access token + refresh token
- Access token stored in httpOnly cookie (7-day expiry)
- Refresh token stored securely, rotated on every use
- Frontend sends access token with every API request in Authorization header
- Backend validates token via Clerk SDK middleware
- Token expiry → Automatic refresh using refresh token

2.2 Role-Based Access Control (RBAC)

Role	Permissions	Scope
User	CRUD own projects, export, share	Own data only
Admin	All user permissions + view all projects, moderate content	Admin
System	Internal API access, cron jobs, background tasks	Service-to-service

2.3 Middleware Implementation

```
// Next.js Middleware – Auth Validation
import { authMiddleware } from "@clerk/nextjs";
export default authMiddleware({
  // Public routes that don't require auth
  publicRoutes: ["/", "/api/curated/platforms", "/api/curated/trending"],
  // Routes that require specific roles
  afterAuth(auth, req, evt) {
    // Check if user is authenticated
    if (!auth.userId || !auth.isPublicRoute) {
      return Response.redirect(new URL("/login", req.url));
    }
    // Admin-only routes
    if (req.nextUrl.pathname.startsWith("/admin")) {
      if (auth.sessionClaims?.role !== "admin") {
        return Response.redirect(new URL("/unauthorized", req.url));
      }
    }
  }
});
export const config = {
  matcher: ["/((?!.*\\.|_next).*)", "/", "/(api|trpc)(.*)"];
};
```

3. Data Protection & Encryption

3.1 Encryption at Rest

- PostgreSQL: AES-256 encryption for sensitive fields (email, API keys)
- Redis: Encrypted connections via TLS
- File Storage: Encrypted S3/R2 buckets for exported files
- Backups: Encrypted backup snapshots with 30-day retention

3.2 Encryption in Transit

- All API calls over HTTPS only (TLS 1.3)
- HSTS headers enforced
- Secure cookie flags: httpOnly, secure, sameSite=strict
- WebSocket connections encrypted via WSS

3.3 API Key Security

OpenAI API keys stored in environment variables only. Never exposed to client. Server-side proxy for all AI API calls. Key rotation every 90 days. Usage monitoring per user.

4. Input Validation & Sanitization

All user inputs validated via Zod schemas. SQL injection prevented via Prisma ORM (parameterized queries). XSS prevention via React's built-in escaping + DOMPurify for HTML content. CSRF protection via SameSite cookies.

```
// Zod Validation Schema Example import { z } from "zod"; const generateIdeasSchema = z.object({ input: z.string().min(10).max(2000), platform: z.enum(["WEB", "MOBILE", "AI_ML", "BLOCKCHAIN", "IOT", "DESKTOP", "EXTENSION", "API_BACKEND"]), userProfile: z.object({ skillLevel: z.enum(["BEGINNER", "INTERMEDIATE", "ADVANCED"]), budget: z.number().min(0).max(1000000), teamSize: z.number().min(1).max(100), timeline: z.string().max(50), techPrefs: z.array(z.string()).max(10) }) }); // Usage in API route export async function POST(req: Request) { const body = await req.json(); const validated = generateIdeasSchema.parse(body); // Throws on invalid // Process validated data... }
```

5. Rate Limiting & Abuse Prevention

Endpoint	Free Tier	Pro Tier	Enterprise	Window
/api/projects	10/hour	100/hour	Unlimited	1 hour
/api/generate	5/hour	50/hour	500/hour	1 hour
/api/evolve	3/hour	30/hour	300/hour	1 hour
/api/analyze	3/hour	30/hour	300/hour	1 hour
/api/export	10/day	100/day	Unlimited	1 day
/api/search	20/min	200/min	2000/min	1 min

5.1 Rate Limiting Implementation

```
// Redis-based Rate Limiting import { Ratelimit } from "@upstash/ratelimit"; import
{ Redis } from "@upstash/redis"; const ratelimit = new Ratelimit({ redis:
Redis.fromEnv(), limiter: Ratelimit.slidingWindow(5, "1 h"), // 5 requests per hour
analytics: true }); export async function middleware(req: NextRequest) { const ip =
req.ip ?? "127.0.0.1"; const { success, limit, remaining } = await
ratelimit.limit(ip); if (!success) { return new Response("Rate limit exceeded", {
status: 429 }); } // Add rate limit headers const res = NextResponse.next();
res.headers.set("X-RateLimit-Limit", limit.toString());
res.headers.set("X-RateLimit-Remaining", remaining.toString()); return res; }
```

6. API Security

- Authentication: JWT tokens via Clerk, validated on every request
- Authorization: RBAC checks in API routes, users can only access own data
- Input Validation: Zod schemas for all request bodies
- Output Sanitization: No sensitive data in responses (no API keys, no passwords)
- CORS: Whitelist-only origins, no wildcard allowed
- Error Handling: Generic error messages, detailed logs server-side only
- Logging: All API calls logged with user ID, timestamp, endpoint, status

7. Third-Party Integration Security

OpenAI API

Server-side only. No client-side API key exposure. Request/response logged for audit. Usage limits per user. Fallback to cached responses.

GitHub API

OAuth tokens stored encrypted. Scope-limited access (read-only for public repos). Token expiry handling.

Clerk Auth

SOC 2 Type II compliant. JWT signature verification. Session management via secure cookies.

Vercel Hosting

DDoS protection via Cloudflare. Automatic HTTPS. Edge network for global distribution.

Railway/Supabase DB

Encrypted at rest. TLS connections. Automated backups. IP allowlisting.

8. Compliance & Privacy

GDPR (EU)

Right to access, Right to deletion, Right to portability, Consent management, Data processing agreements with vendors.

CCPA (California)

Opt-out of data sale (not applicable — no data selling), Disclosure of data collection, Deletion requests.

Data Retention

User data: Until account deletion. Projects: 6 months after last activity (then archived). Logs: 30 days. Analytics: 90 days.

Privacy Policy

Clear disclosure of: What data collected, Why collected, How used, Who shared with, User rights, Contact info.

9. Incident Response Plan

- Detection: Automated alerts via Sentry + UptimeRobot + custom monitoring
- Assessment: Severity classification (P1-Critical, P2-High, P3-Medium, P4-Low)
- Containment: Immediate isolation of affected systems. Rate limiting activation.
- Investigation: Log analysis, impact assessment, root cause identification
- Remediation: Patch deployment, data recovery if needed, user notification
- Post-Incident: Retrospective, documentation update, preventive measures

10. Security Checklist

- ■ HTTPS enforced on all endpoints
- ■ JWT tokens with secure, httpOnly, sameSite=strict cookies
- ■ Input validation via Zod on all API routes
- ■ SQL injection prevention via Prisma ORM
- ■ XSS prevention via React escaping + DOMPurify
- ■ CSRF protection via SameSite cookies
- ■ Rate limiting per user + per IP
- ■ API keys stored in environment variables only
- ■ Sensitive data encrypted at rest (AES-256)
- ■ Role-based access control implemented
- ■ Audit logging for all data access
- ■ Error messages don't leak sensitive info
- ■ CORS whitelist configured
- ■ Security headers (HSTS, CSP, X-Frame-Options)
- ■ Dependency scanning (npm audit, Snyk)
- ■ Penetration testing before production

— End of Security & Access Document —